

Programação Orientada a Objetos

Resumo Completo para Prova

Prof. Rodrigo Martins Pagliares

20 questões · 0,5 pt cada · Múltipla escolha

15%	Diagrama de Sequência do Sistema	2	10%	Modelos de
-----	----------------------------------	---	-----	------------

■ Princípios GRASP

GRASP (General Responsibility Assignment Software Patterns) são 9 princípios que guiam a atribuição de responsabilidades a classes e objetos no design OO. Cada questão da prova tende a apresentar um cenário e perguntar qual princípio se aplica — ou qual decisão viola/respeita um princípio.

1. Information Expert

Definição: Atribuir a responsabilidade à classe que possui a informação necessária para cumpri-la.

Quando usar: Sempre que precisar decidir QUEM executa uma operação.

Exemplo: Venda calcula o total da venda pois possui os LinhasDeltem com preços e quantidades.

Contra-exemplo: Atribuir o cálculo do total a uma classe Caixa, que não tem os itens.

■ **Dica de prova:** Information Expert é o princípio mais fundamental e frequentemente o ponto de partida para qualquer decisão de design.

2. Creator

Definição: B deve criar instâncias de A se: B agrega ou contém A · B usa A · B possui dados iniciais de A · B registra A.

Quando usar: Ao decidir qual classe deve instanciar outra.

Exemplo: Venda cria LinhaDeltemDeVenda (agrega, contém e tem dados para inicializar).

Dica de prova: Geralmente há 2 candidatos plausíveis — escolha o que AGREGA ou CONTÉM.

■ **Dica de prova:** Se B agrega/contém A, B é o criador natural. Não confundir com Controller.

3. Controller

Definição: Atribuir a responsabilidade de receber e tratar eventos de sistema a um objeto que representa: (a) o sistema como um todo (fachada) ou (b) um caso de uso (controlador de caso de uso).

O que NÃO é: Objetos de interface (UI) não são Controllers — eles só exibem/capturam dados.

Exemplo fachada: Sistema (representa todo o sistema como uma caixa preta)

Exemplo caso de uso: GerenciadorDeReservas (para o caso de uso Gerenciar Reserva)

■ **Dica de prova:** Controller desacopla a UI da lógica de negócio — suporta Indirection e Low Coupling.

4. Low Coupling

Definição: Atribuir responsabilidades de modo a minimizar dependências entre classes. Classe com alto acoplamento depende de muitas outras — difícil manter, testar e reusar.

Boa prática: Preferir soluções que não aumentem acoplamento desnecessariamente.

Relação: Indirection cria intermediários que reduzem acoplamento direto.

Atenção: Acoplamento zero é impossível — o objetivo é minimizar.

■ **Dica de prova:** Sempre avalie: essa decisão aumenta dependências sem necessidade?

5. High Cohesion

Definição: Atribuir responsabilidades de modo que a classe permaneça focada, gerenciável e compreensível. Classe com baixa coesão faz coisas não relacionadas.

Exemplo ruim: Classe Venda que também controla interface, acessa banco e envia e-mail.

Exemplo bom: Venda só gerencia itens e total. Outras classes cuidam do resto.

Relação: Pure Fabrication cria classes artificiais para manter coesão.

■ **Dica de prova:** Low Coupling e High Cohesion são forças complementares — decisões de design equilibram ambos.

6. Polymorphism

Definição: Quando variações de comportamento dependem do tipo, atribuir a responsabilidade às subclasses usando polimorfismo — em vez de usar if/else ou switch verificando o tipo.

Problema que resolve: if (tipo == 'A') ... else if (tipo == 'B') ... — código frágil.

Solução: Interface ou classe abstrata com @Override em cada subclasse.

Exemplo: Pagamento (abstract) com subclasses PagamentoDinheiro, PagamentoCartao.

■ **Dica de prova:** Sempre que ver verificação de tipo com if/else, considere Polymorphism.

7. Pure Fabrication

Definição: Criar uma classe que NÃO representa um conceito do domínio real, com o único propósito de manter alta coesão e baixo acoplamento.

Quando usar: Quando nenhuma classe do domínio é boa candidata sem violar coesão.

Exemplo: RepositorioDeVenda — não existe no mundo real, mas organiza acesso a dados.

Distinção: Pure Fabrication é artificial; Information Expert usa classes do domínio.

■ **Dica de prova:** Muito usado para acesso a banco de dados, serviços externos, utilitários.

8. Indirection

Definição: Atribuir responsabilidade a um objeto intermediário para desacoplar dois componentes que não devem depender diretamente um do outro.

Exemplo: Controller entre UI e domínio: UI → Controller → Venda.

Relação: Indirection suporta Low Coupling e Protected Variations.

Atenção: Toda Pure Fabrication é um tipo de Indirection, mas não vice-versa.

■ **Dica de prova:** Padrões como MVC são aplicações de Indirection em larga escala.

9. Protected Variations

Definição: Identificar pontos de variação ou instabilidade e atribuir responsabilidades para criar uma interface estável ao redor deles.

Princípio base: Aberto para extensão, fechado para modificação (Open/Closed Principle).

Exemplo: Interface Pagamento isola variações de forma de pagamento do resto do sistema.

Relação: Usa Polymorphism internamente para implementar as variações.

■ **Dica de prova:** Pense: 'o que pode mudar aqui?' — encapsule esse ponto com uma interface.

Relações entre Princípios GRASP

- **Indirection** → suporta **Low Coupling**
- **Pure Fabrication** → aumenta **High Cohesion** e **Low Coupling**
- **Protected Variations** → usa **Polymorphism** internamente
- **Low Coupling** e **High Cohesion** são forças em tensão — decisões de design equilibram ambos
- **Creator** e **Information Expert** frequentemente apontam para a mesma classe

■ Coleções em Java

A Java Collections API fornece estruturas de dados prontas. A prova testa hierarquia, características de cada implementação e as diferenças conceituais entre ordered/sorted, com/sem generics, e em relação a arrays.

Hierarquia da Collections API

Iterable → **Collection** → **List, Set, Queue**

Map NÃO herda de Collection — é uma hierarquia separada!

Interfaces que herdam de Collection: List, Set, SortedSet, Queue, Deque

Interface / Impl.	Ordered?	Sorted?	Dup.?	Null?	Complexidade add/get
ArrayList	✓ (índice)	✗	✓	✓	O(1) amortizado
LinkedList	✓ (inserção)	✗	✓	✓	O(1) inserção, O(n) acesso
HashSet	✗	✗	✗	1×	O(1)
LinkedHashSet	✓ (inserção)	✗	✗	1×	O(1)
TreeSet	✓	✓	✗	✗	O(log n)
HashMap	✗	✗	chave✗	1×	O(1)
LinkedHashMap	✓ (inserção)	✗	chave✗	1×	O(1)
TreeMap	✓	✓	chave✗	✗	O(log n)

Conceitos-chave

ordered = a ordem de iteração é previsível (ex: ordem de inserção). Não implica ordenação por valor.

sorted = os elementos são mantidos em ordem definida por regra natural (Comparable) ou Comparator.

Ordered ≠ **Sorted**: LinkedHashSet é ordered mas NÃO sorted. TreeSet é sorted (e por isso também ordered).

Set.add() retorna false se o elemento já existe — sem exceção, sem substituição.

Métodos Map: put(k,v) get(k) containsKey(k) keySet() values() entrySet()

Array vs Collection sem Generics vs Collection com Generics

Array (Tipo[]):

- Tamanho FIXO — deve ser definido na declaração
- Acesso O(1) por índice
- Desvantagem principal: tamanho fixo e sem métodos de coleção

Collection<Object> (sem generics):

- Tamanho dinâmico ✓
- Sem segurança de tipo — qualquer Object entra
- Requer cast ao recuperar elementos (ClassCastException em runtime se errado)

Collection<Tipo> (com generics):

- Tamanho dinâmico ✓
- Segurança de tipo em tempo de COMPILAÇÃO ✓
- Elimina casts desnecessários ✓ ← melhor opção

■ Exceções em Java

Hierarquia de Exceções

Throwable

■■■ **Error** (problemas da JVM — nunca capturar)

■ Ex: OutOfMemoryError, StackOverflowError

■■■ **Exception**

■■■ **RuntimeException** (UNCHECKED)

■ Ex: NullPointerException, ArrayIndexOutOfBoundsException

■■■ **Demais subclasses** (CHECKED)

Ex: IOException, SQLException, exceções customizadas

Checked (verificada): subclasse de **Exception** (exceto **RuntimeException**).

- DEVE ser declarada com throws OU capturada com catch
- Representa: recurso indisponível, condição necessária ausente
- Compilador EXIGE tratamento

Unchecked (não-verificada): **RuntimeException** ou **Error**.

- NÃO precisa ser declarada nem capturada
- Representa erro de programação (bug)
- **INCORRETO** afirmar que checked não precisam ser especificadas!

Blocos try / catch / finally

finally: SEMPRE executa — com ou sem exceção.

Exceção: System.exit() impede a execução do finally.

Se finally contém return, ele SOBRESCREVE o return do try.

Multicatch (Java 7+):

```
catch(IOException | SQLException e) { ... }
```

try-with-resources (Java 7+):

```
try(InputStream is = new FileInputStream(f)) { ... }
```

Fecha o recurso automaticamente ao sair do bloco.

Criando Exceções Customizadas — Padrão da Prova

```
class ExcecaoHospedeNaoEncontrado extends Exception {  
    private String nome; // 1.2  
    private String sobrenome; // 1.2  
  
    public ExcecaoHospedeNaoEncontrado(String mensagem, String nome, String sobrenome) {
```

```
super(mensagem); // 1.3 — inicializa 'message' da Exception

this.nome = nome;

this.sobrenome = sobrenome;

}

}
```

Lançar:

```
throw new ExcecaoHospedeNaoEncontrado("Hospede nao encontrado", nome, sobrenome);
```

Capturar (1.6):

```
try { // 1.5
    c = hl.pesquisarHospedePeloNome("Florentino", "Ariza");
} catch (ExcecaoHospedeNaoEncontrado e) { // 1.6
    System.out.println("Hospede nao encontrado");
}
```

2.1 — ArrayList com generics (coleção de Reserva):

```
ArrayList<Reserva> listaReservas = new ArrayList<>(); // diamond operator
```

2.2 — Adicionar à coleção:

```
listaReservas.add(reserva);
```


■ DSS · Contrato de Operação · Modelos OO

Diagrama de Sequência do Sistema (DSS) — 2 questões

DSS e Diagrama de Sequência (DS) são conceitos DIFERENTES, embora ambos usem notação UML.

DSS — Diagrama de Sequência do SISTEMA

- Fase: **Análise / Requisitos**
- Sistema = caixa preta (:Sistema)
- Participantes: ator externo + :Sistema
- Mostra O QUE o sistema faz
- Operações de sistema são eventos externos
- Artefato de entrada para Contratos

DS — Diagrama de Sequência

- Fase: **Design / Projeto**
- Objetos internos VISÍVEIS
- Participantes: objetos do sistema
- Mostra COMO o sistema faz
- Gerado a partir dos Contratos
- Artefato de entrada para código

Elementos e Notação do DSS

":" + **sublinhado** = instância UML. Ex: :Sistema :Caixa

Seta sólida ■ = chamada de operação (ator → sistema)

Seta tracejada ···■ = valor de retorno (sistema → ator)

loop[guarda] = moldura de iteração com condição booleana

Retorno é opcional — só representa quando o valor importa para o cenário

Tríades → Operações de Sistema

Estilo de escrita de caso de uso que facilita identificar operações:

1. Sistema solicita identificador do item

2. Usuário informa o identificador e quantidade → `entrarItem(idItem, qtd)`

3. Sistema exibe descrição e total parcial

Regra: **Nº de tríades = Nº de operações de sistema**

O passo do meio (Usuário informa) sempre gera uma operação de sistema.

Regras para Nomear Operações de Sistema

✓ `entrarItem(idItem, qtd)` — verbo genérico, parâmetros = info do usuário

✓ `criarNovaVenda()` ✓ `finalizarVenda()` ✓ `fazerPagamento(quantia)`

✓ Bool → forma de pergunta: `foiVendaFinalizada()`

✗ `escanear(idItem)` — **sugere implementação tecnológica**

✗ `processarItem()` / `tratarItem()` / `fazerItem()` — **verbos vagos**

✗ `setItem()` / `getItem()` — **padrão de atributos, não operações de sistema**

✗ **Mesmo parâmetro em mais de uma operação do mesmo caso de uso**

X Parâmetros sem correspondência com dados informados pelo usuário

Fluxo de Artefatos no PU

Modelo de Domínio → DSS → Contratos de Operação → Modelo de Projeto (DS) → Código

Cada seta representa uma transformação: análise → design → implementação.

Contrato de Operação — 1 questão

Um Contrato de Operação define formalmente o que uma operação de sistema faz — serve de ponte entre o DSS (o QUE) e o Diagrama de Sequência de design (o COMO).

Seções do Contrato:

Operação: assinatura completa — `entrarItem(idItem: ItemID, qtd: Inteiro)`

Responsabilidades: descrição em linguagem natural do que a operação faz

Pré-condições: estado ANTES — assumido verdadeiro sem verificação

Pós-condições: estado APÓS — descrito em VOZ PASSADA

Exemplos de pós-condições corretas (voz passada):

- Uma instância de `LinhaDeItemDeVenda` **foi criada**
- A `LinhaDeItemDeVenda` **foi associada** à Venda corrente
- O atributo quantidade da `LinhaDeItemDeVenda` **foi modificado** para `qtd`
- A quantidade de estoque do item **foi decrementada**

Modelos de Análise e Design OO — 2 questões

Análise OO — WHAT

- Encontrar e descrever conceitos no domínio do problema
- Produto: Modelo de Domínio
- Classes conceituais (mundo real)
- Sem métodos normalmente
- Substantivos do domínio → classes
- Se X é número/texto → atributo, não classe!

Design OO — HOW

- Definir objetos de software e colaboração
- Produto: Modelo de Projeto (DS, código)
- Classes de software
- Com métodos e atributos de implementação
- Gerado a partir dos Contratos
- UML é ferramenta — não é o objetivo final

Ciladas da prova sobre Modelos OO:

- X 'MD descreve objetos de software' ← ERRADO (descreve conceitos do mundo real)
- X 'Se X é número no mundo real, X é classe conceitual' ← ERRADO (é atributo!)
- X 'É mais importante saber projetar do que entender UML' ← ERRADO
- X 'Design tem ênfase em encontrar conceitos do domínio' ← ERRADO (isso é Análise)

■ Generics · Visibilidade · Datas · Java SE 7

Generics — 1 questão

- ✓ NÃO aceita tipos primitivos — use wrappers: `Integer`, `Double`, `Boolean`
- ✓ Checagem de tipo em **tempo de compilação** (não em runtime)
- ✓ Elimina casts desnecessários — sem risco de `ClassCastException`
- ✓ Permite escrever código reutilizável para vários tipos
- ✓ Operador diamante (diamond): `new ArrayList<>()` — infere o tipo

TYPE ERASURE: a informação do tipo genérico é APAGADA em tempo de execução.

✗ INCORRETO afirmar que 'a definição do tipo genérico é mantida em runtime' ← 4.1d

Declaração correta:

```
List<Reserva> lista = new ArrayList<>();  
Map<String, Hospede> mapa = new HashMap<>();
```

Design para Visibilidade — 1 questão

Para que o objeto A envie uma mensagem ao objeto B, A precisa ter **visibilidade** de B. Há quatro tipos:

1. Visibilidade de Atributo (persistente)

B é um atributo (campo) de A. Disponível em todo o ciclo de vida de A.

```
class Registradora { private Venda vendaAtual; ... }
```

2. Visibilidade de Parâmetro (temporária)

B é passado como parâmetro de um método de A.

```
public void processar(Venda v) { v.calcularTotal(); }
```

3. Visibilidade Local (temporária)

B é declarado como variável local dentro de um método de A.

```
public void processar() { Venda v = new Venda(); v.calcular(); }
```

4. Visibilidade Global (persistente)

B é acessível globalmente (variável estática/singleton). Raro e desaconselhado.

Manipulação de Datas — Ciladas da Prova

Pré-Java 8:

- `Date` — data/hora básica
- `GregorianCalendar` — datas específicas
- `DateFormat` — formatação
- `calendar.add(field, qtd)` — somar

✗ 4.6e: `Calendar` herda de `GregorianCalendar`

ERRADO — é o inverso!

Java 8+ (`java.time`):

- `LocalDate` — só data
- `LocalTime` — só hora
- `LocalDateTime` — data e hora
- `DateTimeFormatter` — formatar
- `LocalDateTime.now()` — atual
- `LocalDate.of(y,m,d)` — específica
- `date.plus(dias)` — somar

✗ 4.7c: `of()` soma dias ← **ERRADO (cria)**

Java SE 7 — Novidades (questão 4.5)

- ✓ **Files**: classe com métodos estáticos para arquivos (`isDirectory`, `createFile`, `deleteIfExists`, `copy`, `move...`)
- ✓ **String em switch**: antes do Java 7 não era possível
- ✓ **Multicatch**: `catch(IOException | SQLException e)`
- ✓ **Literais numéricos com underscore**: `8_000_000`
- ✓ **try-with-resources**: fechamento automático de recursos

✗ 4.5e: `WatchService` usa o design pattern `Adapter` ← **INCORRETO**

`WatchService` monitora diretórios — não usa `Adapter`.

Gabarito das Questões de Múltipla Escolha da Prova 2

- 4.1 (Generics — **INCORRETA**): d — tipo genérico NÃO é mantido em runtime (type erasure)
- 4.2 (HashSet — **INCORRETA**): d — HashSet é unordered; LinkedHashMap é ordered
- 4.3 (Exceções — **INCORRETA**): e — checked precisam SIM ser especificadas ou tratadas
- 4.4 (ArrayList — **INCORRETA**): a — ArrayList mantém ordem de iteração pelo índice
- 4.5 (Java SE 7 — **INCORRETA**): e — WatchService não usa Adapter
- 4.6 (Datas pré-Java 8 — **INCORRETA**): e — Calendar não herda de GregorianCalendar (inverso)
- 4.7 (java.time — **INCORRETA**): c — `of()` de `LocalDate` cria data específica, não soma dias
- 4.8 (Modelo de Domínio — **INCORRETA**): e — se X é número/texto, é atributo, não classe
- 4.9 (Análise/Design OO — **INCORRETA**): e — não é 'mais importante projetar do que entender UML'